

공학 학사 학위논문

DCT-Page: Frequency-Domain Page Proxies for
Sparse KV Decoding

2026년 12월

서울대학교 공과대학

전기·정보공학부

김 윤 곤

DCT-Page: Frequency-Domain Page Proxies for Sparse KV Decoding

지도교수 김재준

이 논문을 공학 학사학위 논문으로 제출함.

서울대학교 공과대학

전기·정보공학부

김 윤 곤

김윤곤의 학사 학위 논문을 인준함.

2026년 12월

지도교수 김 재 준 (인)

ABSTRACT

Keywords: Long-context Language Model, Sparse Attention, KV Cache, Discrete Cosine Transform, Page Attention

Contents

Abstract	i
1 Introduction	2
1.1 Motivation	2
1.2 Key Insight: Attention Keys are Low-Frequency	2
1.3 Contributions	2
1.4 Thesis Organization	2
2 Background and Related Work	3
2.1 Autoregressive Decoding and the KV-Cache Bottleneck	3
2.2 Sparse and Compressed KV Attention	3
2.2.1 Quest	3
2.2.2 SeerAttention-R	3
2.2.3 InfLLM	3
2.2.4 Multipole Attention	3
2.3 Frequency-Domain Signal Compression	3
3 Method: DCT-Page	4
3.1 Decode-Time Page Layout	4
3.2 Key Insight: Low-Frequency Structure of Attention Keys	5
3.3 DCT-lowpass-IDCT Page Proxy	6
3.4 Scoring and Top-k Selection	8
3.5 Implementation on a Paged-Attention Runtime	9
3.6 Configuration Summary	10
4 Experimental Setup	12
4.1 Models	12
4.2 Benchmarks	12
4.2.1 RULER	12
4.2.2 LongBench v1 and v2	13
4.3 Baselines and Hyperparameters	13
4.4 Speed-Measurement Protocol	14
5 Results	17
5.1 Long-Context Accuracy: RULER	17
5.2 Long-Context Accuracy: LongBench v1 and v2	17

5.3	Decode Throughput and Per-Stage Profile	18
5.4	Comparison with Cluster-Based Multipole Attention	19
5.5	Ablations	19
6	Analysis	22
6.1	Attention-Mass Recall vs. Selector	22
6.2	Scoring-Method Comparison Against Oracle	22
6.3	Oracle Upper-Bound at Fixed Token Budget	22
6.4	Limitations	22
7	Conclusion	23
	Bibliography	24
	초 록	25

List of Tables

3.1	Default DCT-Page configuration used in this thesis.	11
4.1	Per-method configuration used in the experiments.	14
5.1	RULER per-task accuracy at 32K context.	17
5.2	LongBench v1 per-task scores.	18
5.3	LongBench v2 accuracy.	18
5.4	Per-stage decode-time attribution.	20
5.5	DCT-Page vs. Multipole: accuracy and throughput.	20
5.6	Ablation: page size P	20
5.7	Ablation: selected-token budget.	20
5.8	Ablation: compression ratio C/P	21
5.9	Ablation: scoring and GQA-group aggregators.	21

List of Figures

3.1	Decode-time KV page layout used by DCT-Page.	4
3.2	Per-bin DCT energy of KV-cache key pages.	6
3.3	Decode-time pipeline composed on top of upstream FlashInfer.	10
5.1	Decode speedup of DCT-Page over Full-KV.	19

Chapter 1 Introduction

1.1 Motivation

1.2 Key Insight: Attention Keys are Low-Frequency

1.3 Contributions

1.4 Thesis Organization

Chapter 2 Background and Related Work

2.1 Autoregressive Decoding and the KV-Cache Bottleneck

2.2 Sparse and Compressed KV Attention

2.2.1 Quest

2.2.2 SeerAttention-R

2.2.3 InfLLM

2.2.4 Multipole Attention

2.3 Frequency-Domain Signal Compression

Chapter 3 Method: DCT-Page

DCT-Page modifies only the *decode* path of attention. During prefill the model runs unchanged, producing an ordinary key–value (KV) cache. At decode time the cache is reinterpreted as a sequence of fixed-size pages, each page is summarized by a short low-frequency proxy, and the query attends to a constant-size subset: the sink, the top- k pages by proxy score, and a small recent window. The remaining pages are skipped entirely. This chapter describes the layout, the proxy, the selection rule, and the fused kernels that turn the resulting sparsity into measured throughput.

3.1 Decode-Time Page Layout

Let L denote the current KV-cache length and P the page size (default $P=32$). Let S and R be the number of sink and full recent pages, respectively. The cache is partitioned as

$$\underbrace{[0, SP)}_{\text{sink}} \parallel \underbrace{[SP, SP+NP)}_{\text{middle pages}} \parallel \underbrace{[SP+NP, L)}_{\text{recent}}, \quad (3.1)$$

where

$$N = \left\lfloor \frac{L - SP - RP}{P} \right\rfloor \quad (3.2)$$

is the number of *middle* pages, i.e. the pageable region from which DCT-Page selects. The recent zone consists of R full pages plus a partial *open* page that absorbs the most recent $0, \dots, P-1$ tokens; the open page is always attended in full. Figure 3.1 shows the layout for $S=1$, $R=4$, the defaults used throughout this thesis.

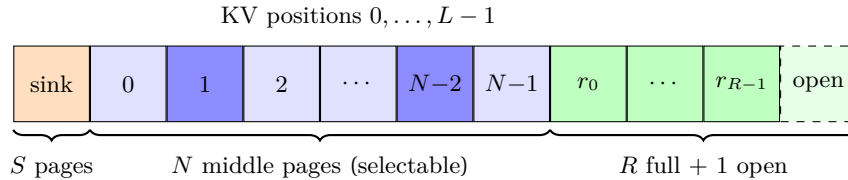


Figure 3.1: Decode-time KV layout. The sink and recent zones are always attended; only the N middle pages enter the top- k selection. Darker blue cells indicate two example selected pages.

Sink and recent zones. The sink absorbs the initial-token attention sinks observed in autoregressive decoder transformers and is kept verbatim to preserve position-aware behaviors learned during pretraining. The recent window protects the most local context, which dominates the output for generation tasks and is cheap to keep at full resolution.

Page granularity. The page size is the fundamental trade-off knob. A smaller P gives finer selection but raises bookkeeping cost (more pages, more scores, more top- k work). A larger P amortizes overhead but loses selectivity within the page. We use $P=32$ throughout, which is also the granularity at which contemporary sparse-attention systems operate (Quest, SeerAttention-R, Multipole).

Fallback below short context. When L is short, the savings from sparsification are dominated by the bookkeeping overhead of scoring and assembly. DCT-Page therefore falls back to standard dense decode attention when $L < L_{\min}$, with $L_{\min}=8192$ tokens by default.

3.2 Key Insight: Low-Frequency Structure of Attention Keys

The starting point for the rest of this chapter is a single empirical observation about decoder-transformer key vectors: viewed as a signal along the sequence axis, they are overwhelmingly low-frequency.

Concretely, take a page of $P=32$ consecutive keys $K_p \in \mathbb{R}^{P \times d}$ from a single (batch, layer, head) triple and apply the orthonormal type-II discrete cosine transform along the sequence axis,

$$\hat{K}_p[m, c] = \sqrt{\frac{2}{P}} \alpha_m \sum_{n=0}^{P-1} K_p[n, c] \cos\left(\frac{\pi(2n+1)m}{2P}\right), \quad m = 0, \dots, P-1, \quad (3.3)$$

with $\alpha_0 = 1/\sqrt{2}$ and $\alpha_m = 1$ for $m \geq 1$. Here $\hat{K}_p \in \mathbb{R}^{P \times d}$ is the frequency-domain representation of the page: row index m runs over DCT bins from low ($m=0$, the DC term) to high frequency, and each column c holds the spectrum of one of the d feature channels. By Parseval’s theorem (DCT is orthonormal), the total energy of K_p equals the sum of squared coefficients $\sum_m \|\hat{K}_p[m, :]\|^2$. On a RULER NIAH-MultiKey 3 trace at 32K context with Llama-3.1-8B-Instruct, the first four DCT bins ($m = 0, 1, 2, 3$) capture, on average, $\approx 85\%$ of the total per-page key energy

across layers, and the first eight bins reach $\approx 90\%$; the remaining high-frequency tail contributes negligibly (Figure 3.2).

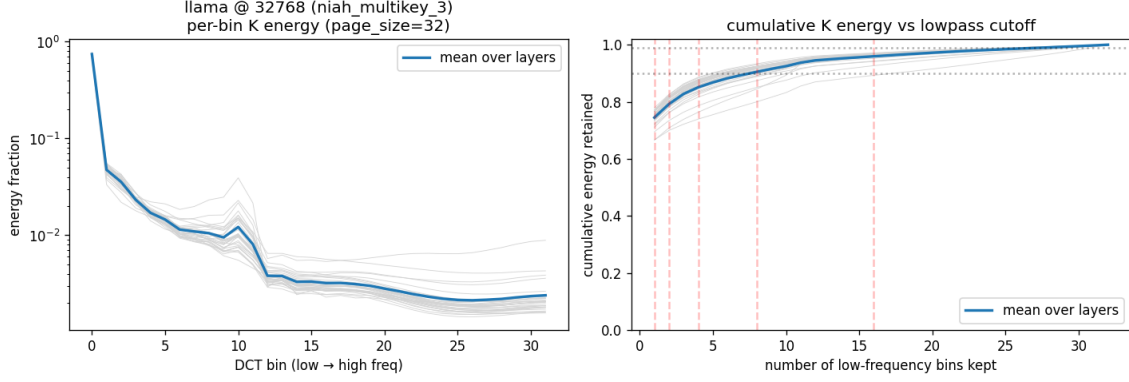


Figure 3.2: Per-bin DCT energy and cumulative energy of decoder-transformer key pages on Llama-3.1-8B-Instruct, page size $P=32$, RULER NIAH-MultiKey 3 at 32K context. Left: per-bin energy fraction; each thin grey line is a single transformer layer, the thick blue line is the across-layer mean. The spectrum is strongly low-pass. Right: cumulative fraction of energy retained as a function of the lowpass cutoff; the first four bins already cover $\approx 85\%$ of the per-page energy and eight bins cover $\approx 90\%$.

This concentration has a direct consequence for sparse attention. The attention score that determines whether page p should be selected for a query q is the inner product $q^\top K_p[n, \cdot]$ for some token n in the page. If K_p itself lives in a low-rank low-frequency subspace, then a DCT-lowpass reconstruction of K_p should be a near-lossless proxy for the selection score, even after collapsing P tokens into a handful of representative tokens. The next section formalizes this proxy and quantifies how well it routes queries to the right page.

3.3 DCT-lowpass-IDCT Page Proxy

Section 3.2 established that a page of keys $K_p \in \mathbb{R}^{P \times d}$ is concentrated in its leading DCT bins. DCT-Page exploits this by replacing each page with a much shorter sequence that retains only those bins.

Fix a target proxy length C with $1 \leq C \leq P$ (“compression ratio” C/P). Let $F \in \mathbb{R}^{P \times P}$ be the orthonormal DCT-II matrix from Eq. (3.3) and let $\text{trunc}_C(\cdot)$ project onto its first C rows.

The DCT-lowpass-IDCT page proxy is

$$K_p^{\text{proxy}} = M K_p \quad \text{with} \quad M = \sqrt{\frac{C}{P}} F^{-1} \begin{bmatrix} I_C & 0 \\ 0 & 0 \end{bmatrix} F \in \mathbb{R}^{C \times P}, \quad (3.4)$$

i.e. apply DCT, keep the leading C coefficients, invert, and rescale by $\sqrt{C/P}$ so that the proxy preserves the energy of the retained band. Crucially M depends only on (P, C) , so it is computed once at start-up and cached as a single $C \times P$ matrix; at runtime, proxy construction is a single batched matrix multiply.

Compression knob. The proxy length C trades selection fidelity against proxy-cache size and per-step compression cost. At $C=1$ the entire page collapses to a single token, which is fast and minimizes memory but discards the higher-frequency structure that distinguishes pages from one another. At $C=4$ the proxy keeps the four energetic bins identified in Figure 3.2, capturing $\approx 85\%$ of the per-page key energy while still compressing the cache by $8\times$. The main configuration in this thesis uses $C=4$ (i.e. a `compress_ratio` of $1/8 = 0.125$); Section 5.5 reports the sweep over $C \in \{1, 2, 4, 8\}$ that justifies this choice.

Incremental update during decode. A decode step appends one token to the KV cache. Until the currently open page accumulates a full P tokens it lives in the recent zone and is attended in full; only when it closes does it become a new middle page and require its proxy to be computed. Consequently at most one new page closes per decode step, and the proxy cache is updated as

$$K_{\text{new}}^{\text{proxy}} = M K_{\text{new}}, \quad M \in \mathbb{R}^{C \times P}, K_{\text{new}} \in \mathbb{R}^{P \times d}. \quad (3.5)$$

This is a single matrix multiply per (batch, kv-head) tuple, costing $O(P \cdot C \cdot d)$ scalar multiplications. The cost is paid *only when a page closes*, i.e. once every P decode steps, so amortized per-step compression work is $O(C \cdot d)$.

After N middle pages have accumulated the proxy cache occupies $O(N \cdot C \cdot d)$ entries per (batch, kv-head). Both the cache size and the per-page compression cost depend only on the number of *closed* middle pages and the page size P , not on the total context length L ; the proxy

machinery therefore imposes no extra context-length scaling on top of the unavoidable $O(L)$ KV cache itself.

3.4 Scoring and Top-k Selection

The proxy cache from Section 3.3 feeds the page-scoring step. Let $Q \in \mathbb{R}^{H_q \times d}$ be the decode query (one token, H_q query heads) and let the model use grouped-query attention (GQA) with H_k key-value heads and $G = H_q/H_k$ query heads per group. Index query heads by (h, j) where h is the KV head and $j \in \{0, \dots, G-1\}$ is the position within the group.

Per-page score. For page p with proxy $K_p^{\text{proxy}} \in \mathbb{R}^{C \times d}$, form the scaled inner products

$$g_{h,j,p,c} = \frac{1}{\sqrt{d}} Q_{h,j}^\top K_p^{\text{proxy}}[c, :], \quad c = 0, \dots, C-1, \quad (3.6)$$

and reduce over the C proxy tokens with a *proxy-token aggregator* $\rho \in \{\text{max}, \text{mean}\}$ and over the G query heads in the GQA group with a *GQA-group aggregator* $\gamma \in \{\text{max}, \text{mean}\}$:

$$S_{h,p} = \gamma_{j=0,\dots,G-1} \left(\rho_{c=0,\dots,C-1} g_{h,j,p,c} \right). \quad (3.7)$$

$S_{h,p}$ is a single scalar measuring how relevant page p is to kv-head h 's decode query. The default configuration is $\rho = \text{max}$ and $\gamma = \text{max}$, which matches a hottest-token / hottest-head heuristic: page p scores high if *any* query head in the group has a large inner product with *any* of its proxy tokens. Section 5.5 reports sensitivity to these choices.

Top- k selection. Let $\mathcal{T}_h \subseteq \{0, \dots, N+R\}$ denote the set of *page indices* that kv-head h will attend to during the current decode step. DCT-Page constructs $\mathcal{T}_h$ as the union of three pieces:

$$\mathcal{T}_h = \underbrace{\{0, \dots, S-1\}}_{\text{sink pages}} \cup \text{TopK}_k(S_{h,\cdot}) \cup \underbrace{\{N, \dots, N+R\}}_{\text{recent + open pages}}, \quad (3.8)$$

where TopK_k returns the k middle-page indices with the largest scores $S_{h,p}$. Note that the top- k set is selected *per kv-head*, so different heads in the same layer may pick different pages.

The decode then performs ordinary scaled-dot-product attention against the keys and values gathered from $\mathcal{T}_h$; the remaining $N - k$ middle pages contribute *nothing* to the output.

To match comparable sparse-attention baselines fairly, we report budgets as the total number of *fixed-size* pages attended per step, denoted B , defined as

$$B = S + k + R, \tag{3.9}$$

the sum of the sink, selected-middle, and full-recent contributions. The selection rule of Eq. (3.8) requires $k \leq N$ — the top- k operator picks k pages out of the N middle pages currently in the cache (Eq. (3.2)); N grows with the context length L while k is a configuration constant. Setting $B=64$ with $S=1, R=4$ therefore fixes $k=59$, and the per-step fixed-size budget is

$$B \cdot P = (1 + 59 + 4) \cdot 32 = 2,048 \text{ tokens},$$

regardless of context length, which is the source of DCT-Page’s near-constant decode cost above the $L_{\min}$ threshold and the figure used by Chapter 5 to match sparse-attention baselines at an equal selected-token budget. The always-attended open page contributes an additional $0, \dots, P-1$ tokens per step depending on the current alignment of the cache; this variable contribution is excluded from B .

Hardware-aware top- k . For $N \leq 1024$ pages, DCT-Page performs the entire top- k in a single fused kernel; for longer contexts it falls back to a two-stage partition-then-merge implementation. Both paths use a parallel bitonic sort that fits the per-head working set in shared memory; this is the dominant decode-time cost saving relative to a CPU- or Python-level top- k launch.

3.5 Implementation on a Paged-Attention Runtime

The decode-time forward of DCT-Page can be viewed as a four-stage pipeline composed on top of an existing paged-attention runtime (FlashInfer [1]). Three of the four stages are scoring/selection operations that FlashInfer does not provide; the last stage is the attention itself. Figure 3.3 summarizes the data flow.

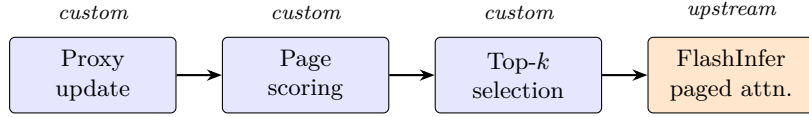


Figure 3.3: DCT-Page decode-time pipeline. Three custom GPU kernels prepare the per-(batch, kv-head) page indices that should be attended; the final attention is dispatched to FlashInfer’s paged-attention primitive over those indices.

Shared paged KV layout. DCT-Page reuses FlashInfer’s paged KV layout, in which the per-layer KV cache is a pre-allocated tensor of fixed-size pages. This layout naturally aligns with the page abstraction of Section 3.1, so no auxiliary indexing structure is required: a page index p refers to the same physical block of P tokens for both proxy construction and attention.

Custom scoring and selection. The proxy update (Eq. (3.5)), page scoring (Eq. (3.7)), and top- k selection (Eq. (3.8)) each run as a single fused GPU kernel. The output of the pipeline is the per-(batch, kv-head) list of page indices $\mathcal{T}_h$. A pure-PyTorch implementation of each stage is retained for correctness checks and for environments without Triton.

Attention via FlashInfer’s paged primitive. Given $\mathcal{T}_h$, the attention itself is dispatched to FlashInfer’s variable-length paged-attention primitive. The primitive accepts a per-(virtual batch, kv-head) index list and computes ordinary scaled dot-product attention against exactly those pages, without materializing the unselected KV. Conceptually, the runtime treats each (b, h) pair as a “virtual batch” with its own page list, which is how DCT-Page realizes per-kv-head heterogeneous selection at hardware speed.

This composition keeps the new code paths small — only the three scoring/selection kernels — while reusing a battle-tested attention primitive for the heavy lifting. Section 5.3 quantifies the resulting throughput.

3.6 Configuration Summary

Table 3.1 lists the default DCT-Page hyperparameters used throughout this thesis; ablations in Section 5.5 sweep (P, B, C, ρ, γ) while holding the rest fixed.

Table 3.1: Default DCT-Page configuration.

Symbol	Meaning	Default
P	Page size (tokens per page)	32
B	Per-step page budget ($S + k + R$)	64
S	Number of sink pages	1
R	Number of full recent pages (excludes the open page)	4
k	Selected middle pages, $= B - S - R$	59
C/P	Compression ratio of the page proxy	1/8
$L_{\min}$	KV length below which decode reverts to dense	8,192
ρ	Proxy-token aggregator	max
γ	GQA-group aggregator	max

Under these defaults, the per-step fixed-size budget evaluates to

$$B \cdot P = \left(\underbrace{1}_S + \underbrace{59}_k + \underbrace{4}_R \right) \cdot 32 = 2,048 \text{ tokens,}$$

which is the selected-token budget used to match sparse-attention baselines in Chapter 5. As noted in Section 3.4, the always-attended open page adds a further $0, \dots, P-1$ tokens per step.

Chapter 4 Experimental Setup

This chapter fixes the models, benchmarks, sparse-attention baselines, and speed-measurement protocol used throughout Chapter 5 and Chapter 6.

4.1 Models

All experiments use two open-weight, decoder-only language models at the $\approx 8\text{B}$ -parameter scale: Llama-3.1-8B-Instruct [2] and Qwen3-8B [3]. Both use rotary positional embedding (RoPE) and grouped-query attention (GQA); Qwen3-8B additionally applies QK-Norm to the query and key projections. All experiments run in `bfloat16` on a single NVIDIA RTX A6000 (48 GiB) unless noted otherwise.

4.2 Benchmarks

DCT-Page is evaluated on three long-context benchmark suites. All prompt-formatting follows the chat-template conventions baked into each model’s tokenizer, with the exceptions documented in Section 4.2.2.

4.2.1 RULER

RULER [4] is a synthetic long-context test suite of 13 tasks, used here at a default context length of 32K tokens. The tasks fall into four families: needle-in-a-haystack retrieval (NIAH Single 1–3, NIAH MultiKey 1–3, NIAH MultiValue, NIAH MultiQuery), variable tracking (VT), word-frequency extraction (Common Words Extraction, Frequent Words Extraction), and long-document question answering (QA-1, QA-2). Each task is scored by its native exact-match or substring metric.

4.2.2 LongBench v1 and v2

LongBench v1 [5] provides a suite of English long-context tasks across single-document QA, multi-document QA, summarization, and few-shot in-context learning. We evaluate on the six tasks most representative of these categories: NarrativeQA, Qasper, MultiFieldQA-en, 2WikiMultihopQA, GovReport, and TriviaQA. Metrics are the official per-task ones (F1 for QA, ROUGE-L for summarization, classification accuracy for in-context learning). TriviaQA is evaluated without a chat template; the other five tasks use the model’s chat format.

LongBench v2 [6] is a 503-question multiple-choice benchmark with four answer options per question. We report overall accuracy and break it down by difficulty (**easy** / **hard**) and by context length (**short** / **medium** / **long**).

4.3 Baselines and Hyperparameters

DCT-Page is compared against five sparse-attention methods organized into two paradigms. All five leave prefill unchanged and modify only decode-time attention; their algorithmic details and provenance are covered in Chapter 2. This section fixes the configuration with which each method is run in Chapter 5.

Page-based baselines. Quest [7], SeerAttention-R [8], and InfLLM [9] share the fixed-page paradigm with DCT-Page. For each we match the *selected-token budget* $B \cdot P = 64 \cdot 32 = 2,048$ tokens per decode step, the same budget DCT-Page uses in Section 3.6; settings within that budget follow each method’s upstream recommendation. Table 4.1 summarizes the per-method configuration.

Cluster-based reference. Multipole Attention [10] organizes keys into a k -means cluster tree rather than fixed-size pages, and its per-step work cannot be matched against the page-based budget in a one-to-one fashion. Multipole typically achieves the highest accuracy across our benchmarks but at substantially longer decode latency, so we exclude it from the main accuracy tables and instead devote §5.4 to its accuracy–throughput trade-off against DCT-Page.

Multipole is run with a single-level configuration of 3.125% leaf-cluster retention — conceptually matched to DCT-Page’s $P=32$ page granularity — and a per-level token-budget threshold of 2,048, with non-selected tokens replaced by their cluster centroid and re-clustering performed at every prefill window.

Full-KV reference. Every accuracy table additionally includes a Full-KV row, which is each model run with its stock dense decode attention. This is the accuracy upper bound that all sparse methods aim to approach at lower decode-time cost.

Model coverage. Page-based baselines are run on whichever model the upstream code supports: SeerAttention-R is reported on Qwen3-8B only (the released gates are trained on the Qwen family, with no equivalent Llama-3 checkpoint), InfLLM on Llama-3.1-8B-Instruct only (the upstream targets the Llama family), and Quest on both models. Multipole and Full-KV are reported on both models.

Table 4.1: Per-method configuration used in the experiments. “Budget” denotes the per-decode-step token budget that the method’s selection rule must satisfy.

Method	Key settings	Models
DCT-Page	$P=32$, $B=64$, $C/P=1/8$, $\rho=\gamma=\max$	both
Quest	page size 32, budget 2,048 tokens	both
SeerAttention-R	token-budget mode, budget 2,048 tokens	Qwen3 only
InfLLM	block size 32, top-64 blocks, sink 32, local 4,096, repr. top-4	Llama only
Multipole	3.125% leaf retention ($\equiv P=32$), threshold 2,048, centroid replacement	both
Full-KV	dense decode attention	both

4.4 Speed-Measurement Protocol

All speed numbers are collected on a single NVIDIA RTX A6000 (48 GiB) from one unified profiler that drives the model with random token inputs and records both the end-to-end decode latency and a per-stage breakdown in a single run. Two execution paths are compared:

- *Full-KV* — the model with its stock dense decode attention, served by upstream FlashInfer’s paged-attention primitive.

- *DCT-Page* — the production configuration introduced in Section 3.5, in which DCT-Page’s scoring/selection kernels feed per-(batch, kv-head) page indices into the same upstream FlashInfer primitive via a virtual-batching scheme that requires no custom fork of the attention runtime.

CUDA graphs. Every speed measurement reported in this thesis uses CUDA-graph capture for the decode loop. This pins the indices/indptr buffers that the paged-attention primitive reads and removes per-step Python and indices-buffer allocation overheads that would otherwise dominate at short context length.

Throughput measurement. For each (L, batch) cell with $L \in \{8, 16, 32, 64, 128\}$ K tokens and batch sizes $\{1, 2\}$, the profiler runs 8 warm-up steps followed by 128 timed decode steps inside the captured CUDA graph. Per-step CUDA-event timings are accumulated and reported as two quantities: (i) *attention-only* latency (the time spent inside the attention sub-module, including scoring/selection for DCT-Page), and (ii) *end-to-end* per-step latency (the same chain extended through the MLP, layer-norm, and projection). Speedups in Section 5.3 are reported as the ratio of the Full-KV latency to the DCT-Page latency at the same (L, batch) .

Per-stage breakdown. Within the same profiler, a chain of CUDA events is placed between the adjacent stages of the decode forward: QKV projection, proxy update, scoring, the fused top- k selection / KV assembly kernel, the attention kernel itself, and the output projection. Because the events are chained end-to-end, the sum of the inter-event elapsed times exactly equals the full decode-step wall clock. The QKV projection and the output projection are identical between Full-KV and DCT-Page — they operate on the query/output token alone and do not depend on the KV-selection strategy — so we exclude them from the comparison reported in Section 5.3. The reported attention-time budget is therefore

$$t_{\text{Full-KV}}^{\text{attn}} \text{ vs. } t_{\text{DCT-Page}}^{\text{proxy}} + t_{\text{DCT-Page}}^{\text{score}} + t_{\text{DCT-Page}}^{\text{topk+assemble}} + t_{\text{DCT-Page}}^{\text{attn}},$$

i.e. the Full-KV attention kernel time against the sum of DCT-Page’s proxy update, scoring, fused top- k /KV-assembly, and attention kernel times. Reporting the comparison as a sum keeps

the overhead of DCT-Page's bookkeeping kernels charged against its measured speedup.

Chapter 5 Results

This chapter reports the empirical results of DCT-Page on the benchmarks and protocol of Chapter 4. Section 5.1 and Section 5.2 compare DCT-Page against page-based sparse-attention baselines on long-context accuracy; Section 5.3 reports decode throughput and a per-stage attribution of the speedup; Section 5.4 examines the accuracy–throughput trade-off against the cluster-based Multipole Attention baseline; and Section 5.5 sweeps the four DCT-Page hyperparameters that most affect accuracy and throughput.

5.1 Long-Context Accuracy: RULER

Table 5.1 reports per-task RULER accuracy at 32K context for the two base models, comparing DCT-Page against the page-based sparse-attention baselines. All sparse methods are run at the $B \cdot P = 2,048$ selected-token budget fixed in Section 4.3; bold entries mark the best sparse-method score per column.

Table 5.1: RULER accuracy on 13 tasks at 32K context, in percent. Models: Llama = Llama-3.1-8B-Instruct, Qwen3 = Qwen3-8B. Sparse methods are matched at a 2,048-token selected budget. Task abbreviations: S = NIAH Single, MK = NIAH MultiKey, MV = NIAH MultiValue, MQ = NIAH MultiQuery, VT = Variable Tracking, CWE = Common Words Extraction, FWE = Frequent Words Extraction, QA = long-document question answering. Best sparse-method score per column in bold.

Model	Method	S1	S2	S3	MK1	MK2	MK3	MV	MQ	VT	CWE	FWE	QA1	QA2	Avg
Llama	Full-KV	100.00	100.00	100.00	100.00	96.00	100.00	98.00	100.00	98.40	48.40	93.33	72.00	48.00	88.78
	Quest	100.00	100.00	100.00	100.00	96.00	32.00	97.00	99.00	97.60	30.00	82.67	72.00	48.00	81.10
	InfLLM	100.00	88.00	100.00	84.00	12.00	8.00	75.00	92.00	44.00	6.40	93.33	32.00	32.00	58.98
	DCT-Page	100.00	100.00	100.00	100.00	96.00	92.00	100.00	100.00	96.00	28.00	93.33	72.00	44.00	86.26
Qwen3	Full-KV	100.00	100.00	100.00	96.00	88.00	96.00	95.00	100.00	96.00	86.40	93.33	56.00	48.00	88.83
	Quest	100.00	100.00	100.00	92.00	88.00	8.00	90.00	90.00	98.40	58.80	84.00	48.00	48.00	77.32
	SeerAttention-R	100.00	84.00	92.00	92.00	40.00	28.00	65.00	88.00	78.40	21.60	90.67	44.00	44.00	66.74
	DCT-Page	100.00	100.00	100.00	96.00	88.00	72.00	97.00	100.00	100.00	46.00	89.33	52.00	48.00	83.72

5.2 Long-Context Accuracy: LongBench v1 and v2

LongBench measures long-context performance on realistic text rather than synthetic retrieval.

Table 5.2 reports per-task scores on the six-task v1 subset (Section 4.2.2); Table 5.3 reports v2

multiple-choice accuracy broken down by difficulty (easy / hard) and by context length (short / medium / long).

Table 5.2: LongBench v1 scores on six representative tasks. Models: Llama = Llama-3.1-8B-Instruct, Qwen3 = Qwen3-8B. Metrics are per-task official metrics (F1 for QA, ROUGE-L for summarization, accuracy for in-context learning). Best sparse-method score per column in bold.

Model	Method	NarrativeQA	Qasper	MFQA-en	2WikiMQA	GovReport	TriviaQA	Avg
Llama	Full-KV	29.24	45.37	54.99	45.33	35.34	91.65	50.32
	Quest	28.99	43.08	51.25	44.80	29.97	91.98	48.35
	InfLLM	25.80	43.89	49.41	46.47	34.70	92.07	48.72
	DCT-Page	29.50	45.96	55.08	44.68	35.04	91.34	50.27
Qwen3	Full-KV	24.98	48.32	51.85	43.14	33.46	90.48	48.70
	Quest	25.47	45.74	50.74	43.15	31.20	90.30	47.77
	SeerAttention-R	25.30	47.13	50.85	43.33	32.54	90.31	48.24
	DCT-Page	24.93	48.01	51.93	42.89	33.54	90.48	48.63

Table 5.3: LongBench v2 multiple-choice accuracy (%) broken down by difficulty and context-length category. Models: Llama = Llama-3.1-8B-Instruct, Qwen3 = Qwen3-8B. Best sparse-method score per column in bold.

Model	Method	Overall	Easy	Hard	Short	Medium	Long
Llama	Full-KV	29.82	30.21	29.58	35.00	26.05	28.70
	Quest	18.29	22.40	15.76	38.33	10.70	0.00
	InfLLM	26.04	25.52	26.37	31.67	26.05	16.67
	DCT-Page	30.02	33.33	27.97	34.44	26.05	30.56
Qwen3	Full-KV	29.64	31.25	28.62	28.70	26.98	33.33
	Quest	18.09	21.88	15.76	38.33	9.77	0.93
	SeerAttention-R	32.60	36.46	30.23	38.89	30.23	26.85
	DCT-Page	32.01	33.33	31.19	38.89	28.84	26.85

5.3 Decode Throughput and Per-Stage Profile

This section reports decode-time speedup of DCT-Page over Full-KV at five context lengths $L \in \{8, 16, 32, 64, 128\}$ K tokens and two batch sizes (1, 2), measured under the protocol of Section 4.4. Figure 5.1 shows the speedup curves; Table 5.4 decomposes the attention-time budget into the per-stage components defined in Section 4.4.

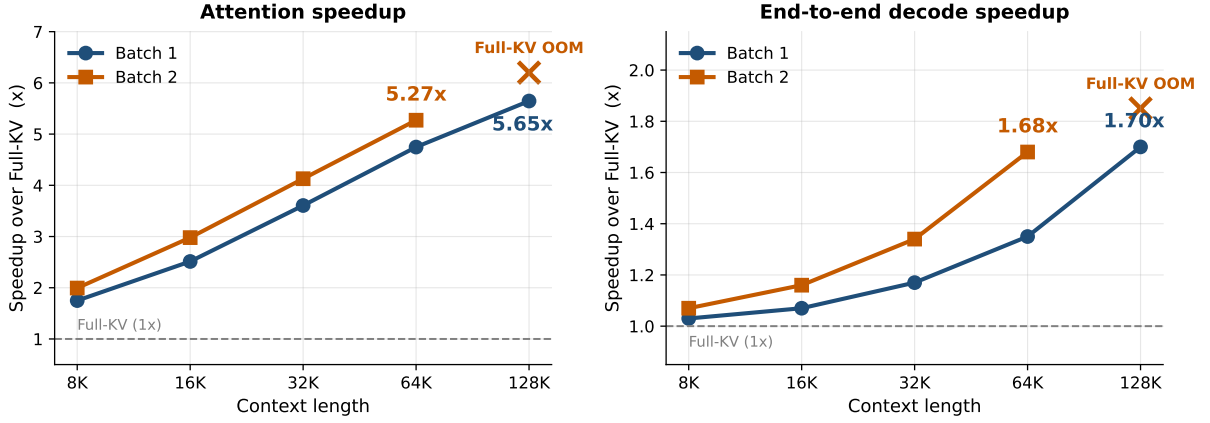


Figure 5.1: Decode speedup of DCT-Page over Full-KV on a single A6000 with Llama-3.1-8B-Instruct. Left: attention-only speedup — up to 5.65 $\times$ at 128K context (batch 1) and 5.27 $\times$ at 64K (batch 2). Right: end-to-end decode speedup including MLP and layer-norm — 1.70 $\times$ at 128K (batch 1) and 1.68 $\times$ at 64K (batch 2). “Full-KV OOM” marks configurations at which the dense baseline exhausts the 48 GiB device memory.

5.4 Comparison with Cluster-Based Multipole Attention

Multipole Attention is omitted from the head-to-head accuracy tables above because its cluster-based decode path lies on a different point of the accuracy–throughput Pareto frontier than the page-based methods (Section 4.3). Table 5.5 reports DCT-Page and Multipole side by side on each accuracy benchmark and on decode throughput, so that the trade-off can be read directly.

5.5 Ablations

We sweep the four DCT-Page hyperparameters that most influence the accuracy–throughput trade-off, holding all others at the defaults of Table 3.1. All sweeps report RULER average accuracy on Llama-3.1-8B-Instruct at 32K context; the conclusions transfer to Qwen3-8B and to LongBench (not shown).

Table 5.4: Per-stage decode-time attribution at $L = 32,768$, batch 1. The Full-KV row reports the time spent inside its attention kernel; the DCT-Page rows split the equivalent attention-time budget into proxy update, page scoring, the fused top- k selection / KV assembly kernel, and the final attention kernel. All values are in milliseconds per decode step, averaged over 128 timed steps. QKV and output projections are identical between the two paths and are excluded.

Stage	Time (ms/step)
Full-KV: attention kernel	6.400
DCT-Page: proxy update	0.004
DCT-Page: page scoring	0.443
DCT-Page: top- k selection + KV assembly	0.312
DCT-Page: attention kernel	0.630
DCT-Page: total	1.389

Table 5.5: DCT-Page vs. Multipole Attention. RULER and LongBench v1 are reported as averages across their respective tasks; LongBench v2 is reported as overall accuracy. Decode throughput is the tokens per second at $L = 32,768$, batch 1, measured under the protocol of Section 4.4.

Method	RULER Avg		LongBench v1		LongBench v2		Throughput (tok/s)
	Llama	Qwen3	Llama	Qwen3	Llama	Qwen3	
DCT-Page	86.26	83.72	50.27	48.63	30.02	32.01	38.52
Multipole	89.19	88.04	50.57	48.48	28.23	31.41	7.50

Table 5.6: Page size P sweep at fixed selected-token budget $B \cdot P = 2,048$. Smaller P gives finer selection at the cost of more bookkeeping work; larger P amortizes overhead but loses selectivity inside the page.

P	B (pages)	RULER Avg (%)	Tok/s @ 32K
16	128		
32	64		
64	32		
128	16		

Table 5.7: Selected-token budget sweep at fixed $P=32$. k is the middle-page count $B - S - R$ at $S=1, R=4$.

Budget (tokens)	B (pages)	k (middle)	RULER Avg (%)
1,024	32	27	
2,048	64	59	
4,096	128	123	

Table 5.8: Compression ratio sweep at fixed $P=32$. C is the number of proxy tokens retained per page (Section 3.3).

C	C/P	RULER Avg (%)
1	1/32	
2	1/16	
4	1/8	
8	1/4	

Table 5.9: Choice of proxy-token aggregator ρ and GQA-group aggregator γ (Section 3.4).

ρ	γ	RULER Avg (%)
max	max	
max	mean	
mean	max	
mean	mean	

Chapter 6 Analysis

6.1 Attention-Mass Recall vs. Selector

6.2 Scoring-Method Comparison Against Oracle

6.3 Oracle Upper-Bound at Fixed Token Budget

6.4 Limitations

Chapter 7 Conclusion

Bibliography

- [1] Z. Ye et al., “FlashInfer: Efficient and customizable attention engine for LLM inference serving,” in *Proceedings of Machine Learning and Systems (MLSys)*, 2025.
- [2] A. Grattafiori et al., *The Llama 3 herd of models*, 2024. arXiv: [2407.21783](#).
- [3] A. Yang et al., *Qwen3 technical report*, 2025. arXiv: [2505.09388](#).
- [4] C.-P. Hsieh et al., “RULER: What’s the real context size of your long-context language models?” In *Proceedings of the Conference on Language Modeling (COLM)*, 2024.
- [5] Y. Bai et al., “LongBench: A bilingual, multitask benchmark for long context understanding,” in *Proceedings of the Annual Meeting of the Association for Computational Linguistics (ACL)*, 2024.
- [6] Y. Bai, S. Tu, J. Zhang, H. Peng, et al., “LongBench v2: Towards deeper understanding and reasoning on realistic long-context multitasks,” in *Proceedings of the Annual Meeting of the Association for Computational Linguistics (ACL)*, 2025.
- [7] J. Tang, Y. Zhao, K. Zhu, G. Xiao, B. Kasikci, and S. Han, “Quest: Query-aware sparsity for efficient long-context LLM inference,” in *Proceedings of the International Conference on Machine Learning (ICML)*, 2024.
- [8] Y. Gao et al., “SeerAttention-R: Sparse attention adaptation for long reasoning,” *arXiv preprint arXiv:2506.08889*, 2025.
- [9] C. Xiao et al., “InfLLM: Training-free long-context extrapolation for LLMs with an efficient context memory,” in *Advances in Neural Information Processing Systems (NeurIPS)*, 2024.
- [10] R. Singh et al., *Multipole attention: Hierarchical clustering for long-context sparse attention*, TODO: replace with actual citation, 2025.

초 록

주요어: 장문맥 언어모델, 회소 어텐션, KV 캐시, 이산 코사인 변환, 페이지 어텐션